

CSA0609 - DESIGN AND ANALYSIS OF ALGORITHMS

First Palindromic String in an Array

1. Aim

To write a Python program to find the **first palindromic string** in an array of strings using an appropriate algorithm and analyze its **time and space complexity**.

2. Algorithm

1. Start with the given array of strings.
2. Traverse each string from left to right.
3. For each string, reverse the string.
4. Compare the original string with its reverse.
5. If both are equal, the string is a palindrome.
6. Return the first palindromic string found.
7. If no palindrome is found, return an empty string.

3. Pseudocode

Algorithm FirstPalindrome(words)

Input: An array of strings words

Output: First palindromic string

1. For each word in words do
2. reverse $\leftarrow$ Reverse(word)
3. If word = reverse then
4. Return word
5. End If
6. End For
7. Return ""
8. End Algorithm

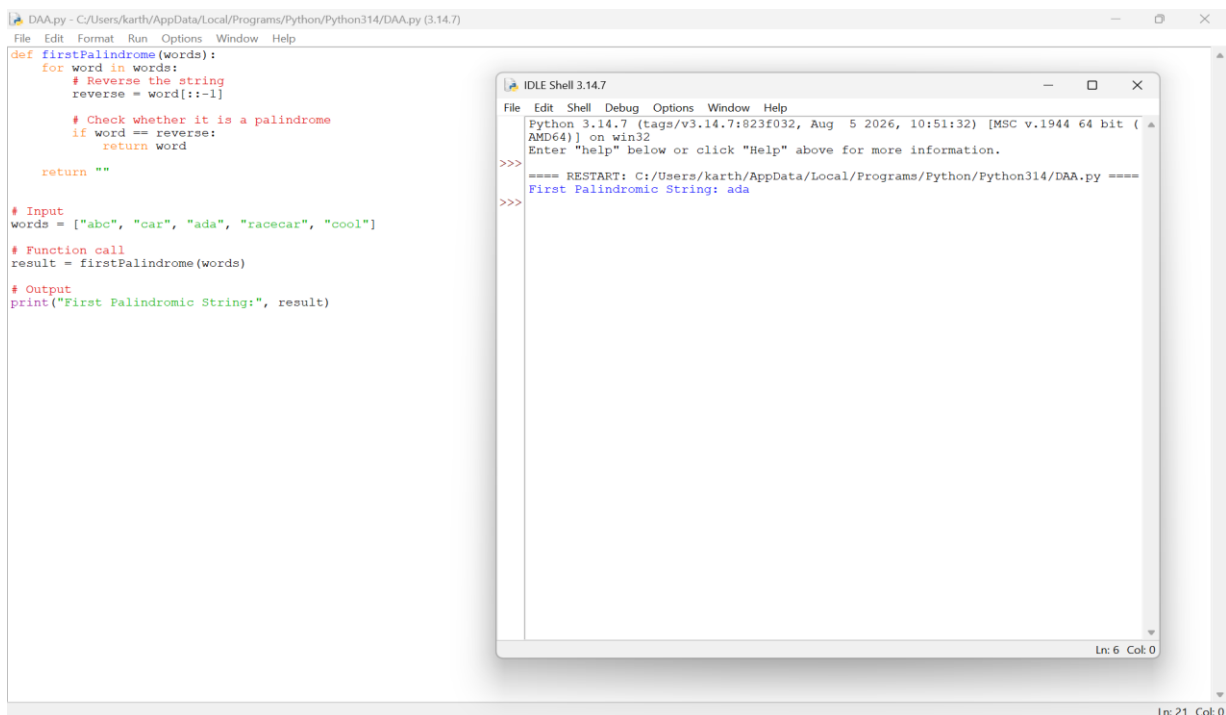
4. Python Code

```
def firstPalindrome(words):  
  
    for word in words:  
  
        # Reverse the string  
        reverse = word[::-1]  
  
        # Check whether it is a palindrome  
        if word == reverse:  
  
            return word  
  
    return ""  
  
# Input  
words = ["abc", "car", "ada", "racecar", "cool"]  
  
# Function call  
result = firstPalindrome(words)  
  
# Output  
print("First Palindromic String:", result)
```

5. Output

First Palindromic String: ada

6. Output Screenshot:



The screenshot displays the Python IDLE environment. The main window shows the Python script 'DAA.py' with the following code:

```
def firstPalindrome(words):  
    for word in words:  
        # Reverse the string  
        reverse = word[::-1]  
        # Check whether it is a palindrome  
        if word == reverse:  
            return word  
    return ""  
  
# Input  
words = ["abc", "car", "ada", "racecar", "cool"]  
  
# Function call  
result = firstPalindrome(words)  
  
# Output  
print("First Palindromic String:", result)
```

An 'IDLE Shell 3.14.7' window is open in the foreground, showing the execution output:

```
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32  
Enter "help" below or click "Help" above for more information.  
>>>==== RESTART: C:/Users/karth/AppData/Local/Programs/Python/Python314/DAA.py ====  
First Palindromic String: ada  
>>>
```

The status bar at the bottom of the IDLE window indicates 'Ln: 6 Col: 0'.

7. Complexity Analysis

Let:

- **n** = number of strings in the array
- **m** = maximum length of a string

Time Complexity

For each string, reversing and comparing takes **O(m)** time.

In the worst case, all n strings are checked.

$$O(n \times m)$$

Space Complexity

The reversed string requires **O(m)** extra space.

$$O(m)$$

Cases

Case	Time Complexity
Best Case	O(m)
Average Case	O(n × m)
Worst Case	O(n × m)
Space Complexity	O(m)

8. Result

Thus, the **First Palindromic String in an Array** was successfully implemented using Python. The program correctly finds the first palindrome, and its **worst-case time complexity is O(n × m)** with **O(m) space complexity**.